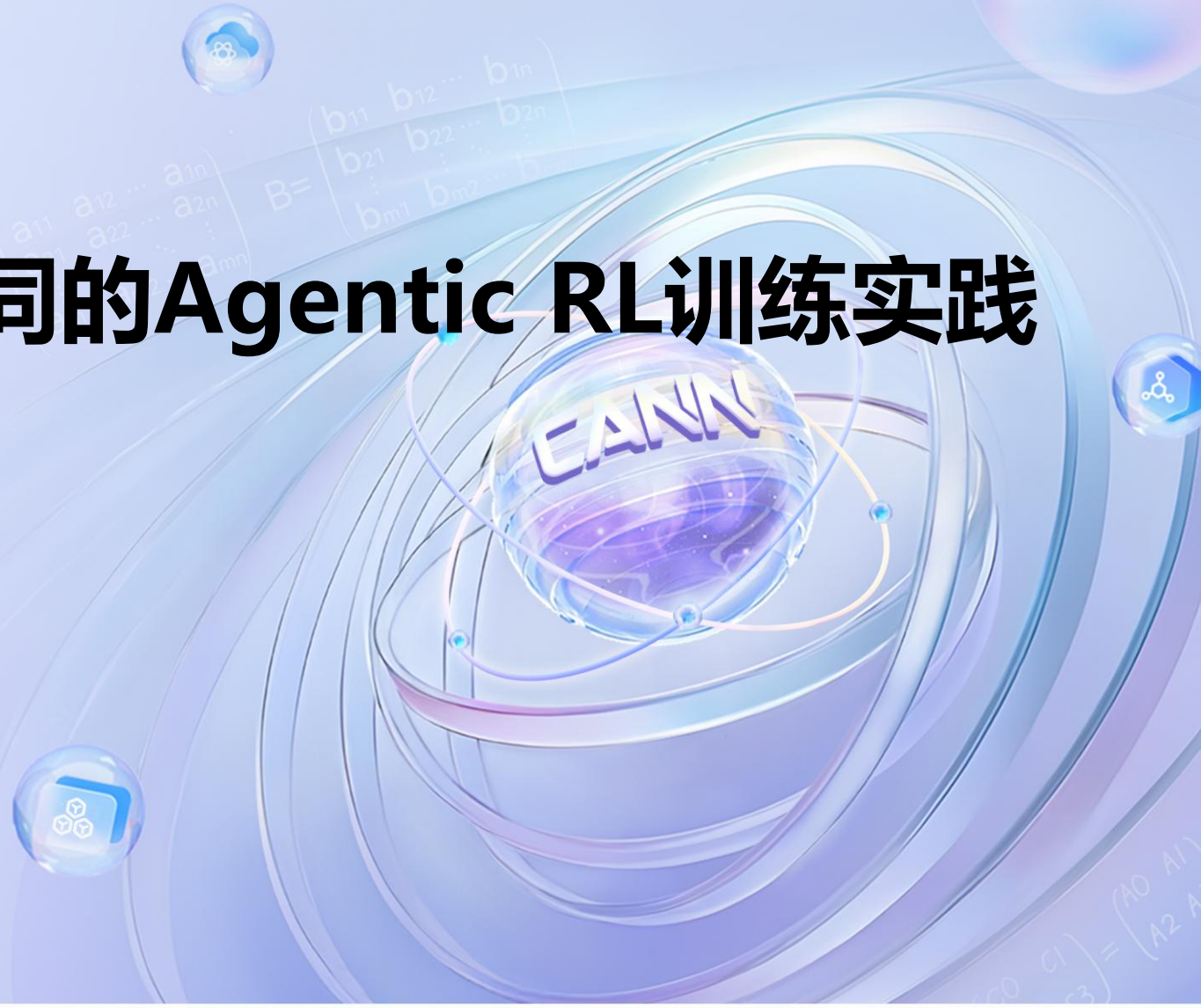
鲲鹏+昇腾+灵衢协同的Agentic RL训练实践

时间：2026/10/19



CANN

目录

Part 1 Agentic RL 介绍和训练流程

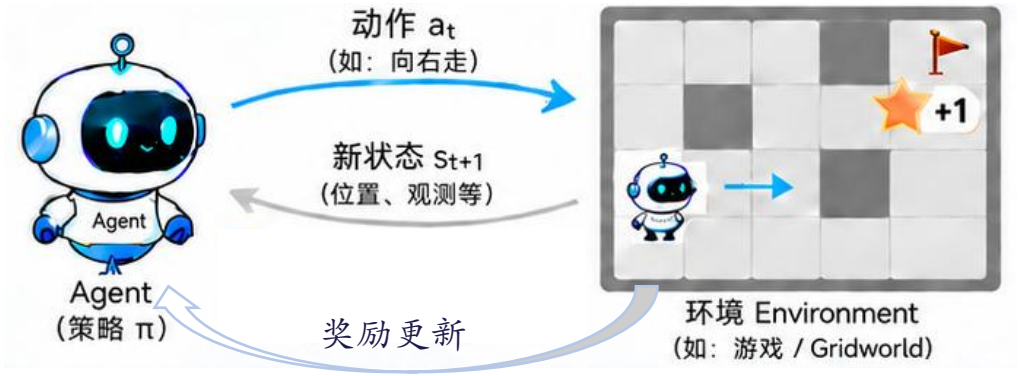
Part 2 AgentENV 在鲲鹏 CPU 上的适配与优化

Part 3 DeepSeek-V4 低精度训练与训推精度对齐方案

Part 4 实验、总结与Future Plan

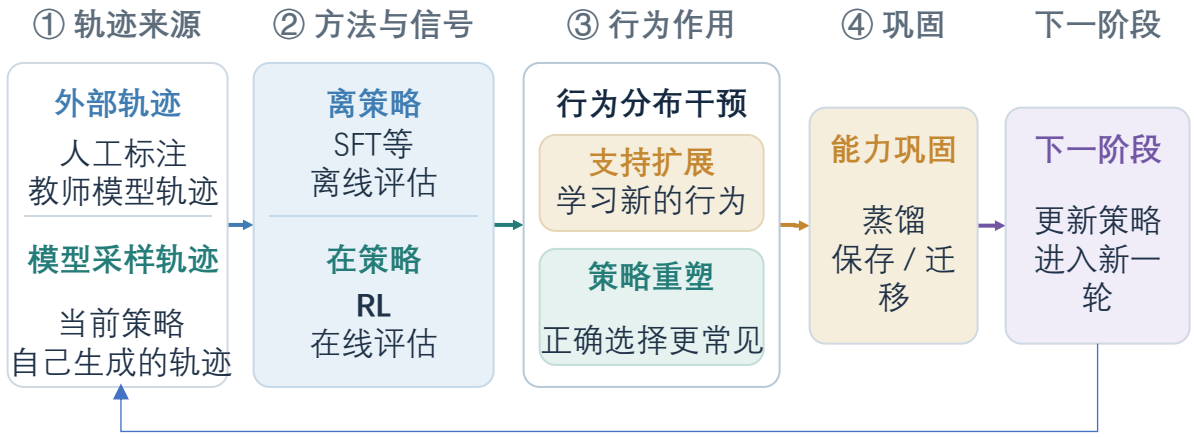
RL（Reinforcement Learning，强化学习）训练

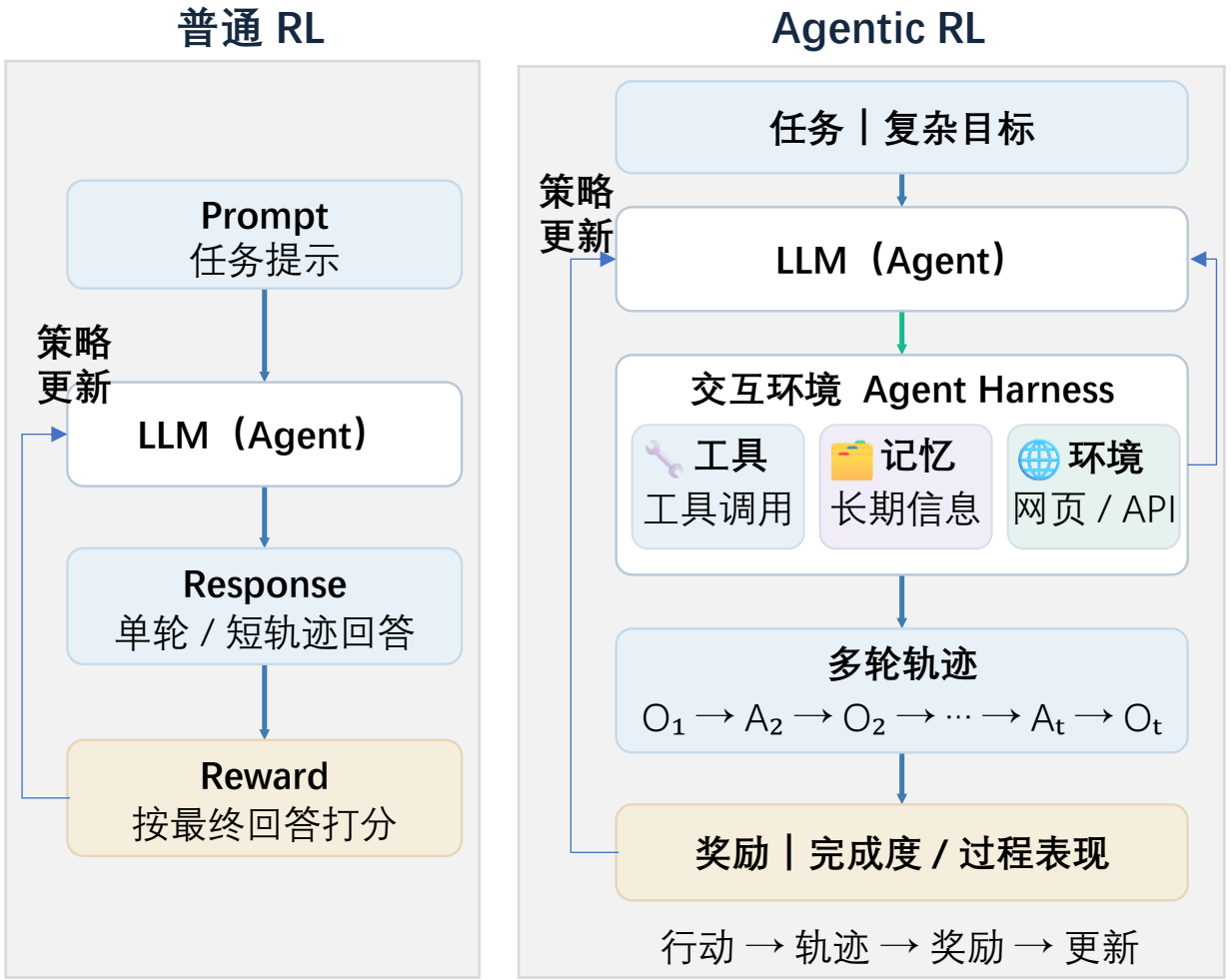
◆ RL 基本框架



◆ LLM 训练中的RL训练

LLM 预训练 -> LLM 后训练（STF、RL、OPD等）。后训练流程：

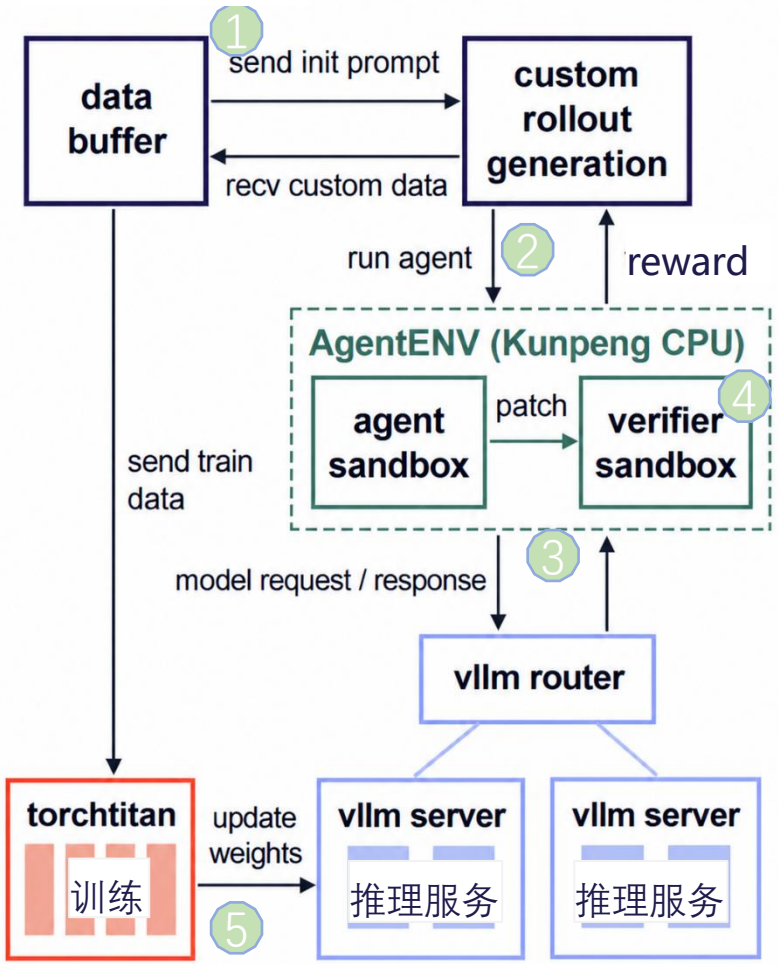




比较维度	LLM RL	Agentic RL
优化对象	单次 / 短轨迹生成	完整任务执行轨迹
交互方式	文本生成	多轮交互、工具与环境
评价目标	最终回答质量	任务整体效果

Agentic RL 训练

◆ RL 框架



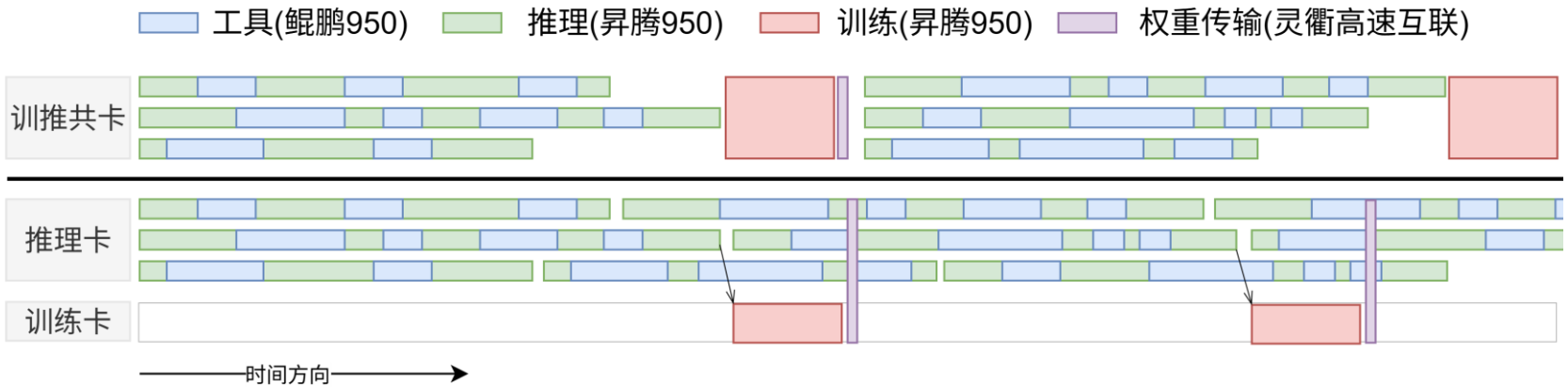
Agentic RL 训练流程

1. **任务下发：** 从数据集选择prompt下发
2. **环境创建：** 创建 agent 沙箱环境
3. **多轮交互：** agent 在沙箱中循环访问推理服务和调用工具
4. **独立评测：** 独立 verifier 沙箱评测，返回 reward
5. **策略更新：** 训练更新策略，向推理侧同步权重

Agentic RL 训练面对的主要挑战

- **沙箱环境：** 大量沙箱并发，高频工具调用，对沙箱环境基础设施提出更高要求。（鲲鹏）
- **内存与通信：** 最长1M长序列和超大规模模型带来的内存与通信挑战。（昇腾+灵衢）
- **训推一致：** RL 训练精度稳定性与训推一致性挑战。（算子）

◆ 部署与运行时序（训推共卡，训推分离）



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

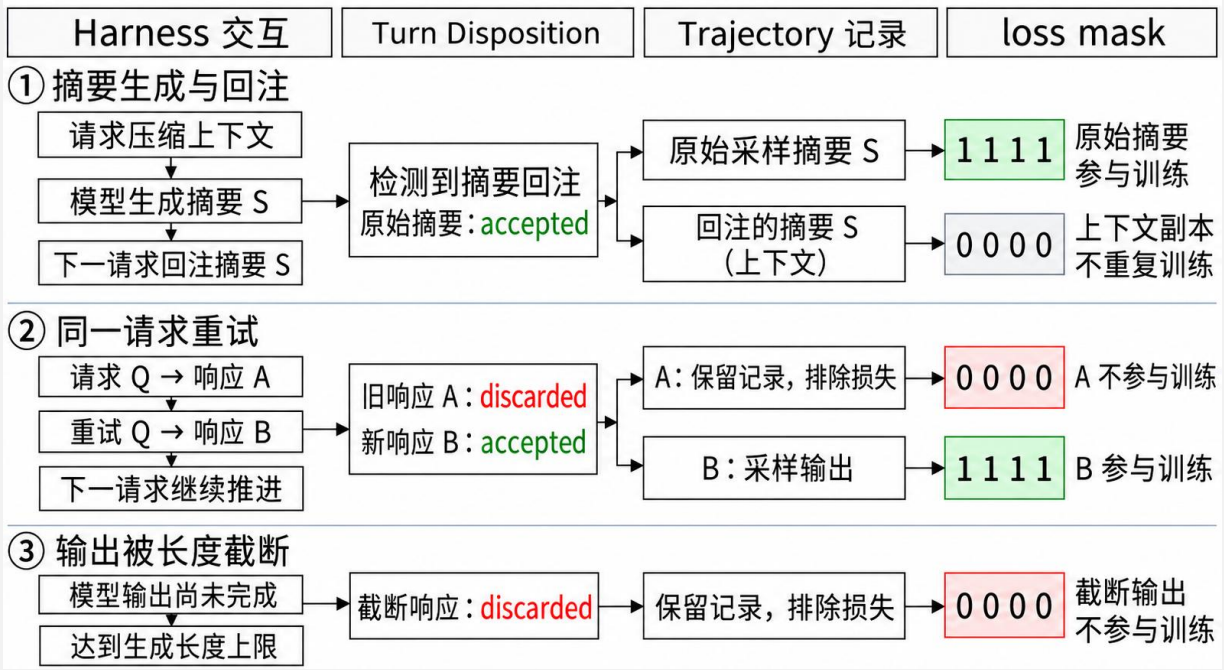
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right] - \beta\mathbb{D}_{KL}\left[\pi_\theta||\pi_{ref}\right]\right\}$$

CANN

沙箱与harness

◆ RL 框架代码提供 harness 能力

- 支持 codex 和 claude code 两种 harness。
- 引入 Turn Disposition，以会话为单位，为每个已生成响应维护状态，并根据后续协议事件确定其训练资格。



◆ 沙箱提供执行环境



◆ 大规模多 Agent RL 对沙箱的需求

需求	具体要求
高并发	数千到数万个沙箱同时运行，彼此独立
强隔离	每个 Agent 独立内核；恶意代码相互不影响
高保真环境	必须是完整 Linux VM；真实内核与网络栈
快速启停	需要快，降低Rollout时间；

◆ 本次实践采用 AgentENV 进行沙箱管理，AgentENV 已完成鲲鹏上的适配与优化

目录

Part 1 Agentic RL 介绍和训练流程

Part 2 AgentENV 在鲲鹏 CPU 上的适配与优化

Part 3 DeepSeek-V4 低精度训练与训推精度对齐方案

Part 4 实验、总结与Future Plan

AgentENV：面向AI Agent的大规模分布式执行环境

AgentENV（AENV）是一个面向 AI Agent 的大规模分布式执行环境平台。

利用 **Firecracker microVM** 为每个 Agent 提供一个独立内核、独立文件系统、独立网络命名空间的微虚拟机沙箱。

满足 Agentic RL 训练、自动化评测与代码执行沙箱场景中**"强隔离，高密度、低成本"**的关键诉求。

鲲鹏 950 服务器原生支持 SMT 超线程，单 CPU 开启后最多可提供 **192 个逻辑核心**，依托 AENV 的 CPU 超分能力，可实现高密度沙箱并发部署。



内核级隔离



毫秒级启停



CPU、内存超配

20x CPU超分

鲲鹏950单CPU支持 SMT 超线程，可提供**192 个逻辑核心**

9.6x 内存超配

来源于生产环境内存超分能力数据

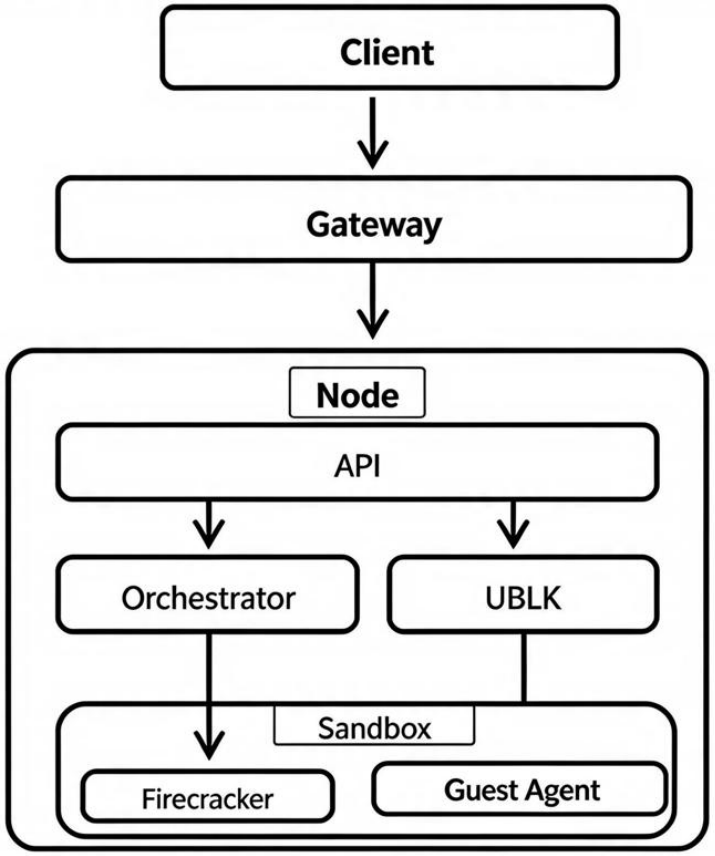
毫秒级启停

鲲鹏950支持DDR5内存条，可达成冷启动/恢复 **<50ms**，暂停/快照 **<100ms**，让"环境"变成可按需分配、瞬时回收的细粒度资源。

https://gitcode.com/cann

CANN

AgentENV 架构总览：节点请求链路



组件	职责
Gateway	系统的统一入口，将流量转发到对应的后端节点。
Node	sandbox的执行单元，每个节点内部有API、Orchestrator和UBLK。API对外暴露接口，Orchestrator管理 sandbox的创建、暂停、恢复和删除，UBLK负责把数据面请求转发到具体 sandbox。
Orchestrator	沙箱生命周期状态机管理。
uvm-ublk-daemon	常驻进程统一管理所有 ublk 设备。
Sandbox	任务实际运行的隔离环境，Firecracker microVM提供强隔离边界，VM内部的 Guest Agent 负责命令执行、文件操作和环境初始化。
Firecracker VM	基于 KVM 的微虚拟机，提供硬件级隔离。

核心能力一：overlaybd 分层块设备 + ublk + io_uring

毫秒级启动

overlaybd：从"文件层分层"到"块层分层"

```
+-----------------------------------+
| upper layer（当前修改）          | ← 每个沙箱独立
+-----------------------------------+
| layer N（OCI 镜像层）            | ← 多沙箱共享、去重
| ...                               |
| layer 0（基础层）                 |
+-----------------------------------+
```

分层共享：基础镜像层多沙箱共享，仅需为每个沙箱保存增量修改，极大降低存储开销

按需加载：overlaybd 支持按需读取块数据，无需完整下载镜像即可启动沙箱

ublk + io_uring：零系统调用开销 I/O

Firecracker VM

↓

/dev/ublkbN（ublk 块设备）

↓

overlaybd 层叠镜像

↓

io_uring（异步 I/O）

高性能 I/O：ublk + io_uring 实现内核态到用户态的高效数据面转发，整条路径几乎无系统调用开销

高性能存储：块设备级别的 I/O 性能，满足 AI 训练与代码执行的高吞吐需求

CANN

核心能力二：Firecracker microVM 内核级隔离

基于ARM的虚拟化扩展能力+开源Firecracker：利用 KVM 硬件虚拟化，给每个执行环境独立的虚拟机，形成沙箱之间的相互隔离。每个沙箱是一台 microVM，Agent 在沙箱内做任何事均不影响宿主机。



硬件级隔离

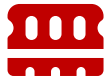
核心能力三：内存超配 9.6x

内存超配与资源回收

AENV 依托 ublk 提供高性能 I/O，同时在**存储与内存快照数据之间共享宿主机页缓存。**

内存膨胀机制（**Memory Ballooning**）将可回收的虚拟机内存交还宿主机。

随着运行时长增加、环境数据产生差异，生产环境中可实现 **9.6 倍** 内存超分。



低内存开销

强大的生态兼容能力



E2B 协议兼容：基于 E2B Sandbox API 协议，完成与 Agentic RL 训练框架的端到端集成

CANN

鲲鹏平台部署条件：Hardware + Software

Hardware

鲲鹏 950 服务器

384 logical CPUs

4 NUMA nodes

1.5T RAM with swap disabled

Software

Linux 6.6+ 内核（ 官网推荐 6.8+）

需要支持/dev/kvm （KVM 硬件虚拟化强要求）

实测 openEuler 24.03（内核 6.6）可用

实测 Ubuntu 24.04（内核 6.8）可用

开源社区

git clone https://github.com/kvcache-ai/AgentENV.git

Rust 实现，支持单节点到 Kubernetes 集群的渐进式部署

注意：openEuler 24.03 内核 6.6 在开启 CONFIG_BLK_DEV_UBLK 后可用。

CANN

PVspinlock 实践优化：1:2 超分场景性能提升 11.66%

问题背景

KVM 虚拟化存在 Host 和 Guest 两层调度，两者互不感知。

vCPU 是主机线程受 Host 调度；虚拟机内线程受 Guest 调度。

当 vCPU 超分时，若持有 spinlock 的 vCPU 被调度出去，其他 vCPU 只能无意义自旋等待，导致抢锁开销激增，表现为虚拟机 CPU 冲高、Host 整机性能下降。

解决方案：PVspinlock

openEuler OLK-6.6 合入 PVspinlock，在 ARM 上对spinlock进行了优化。

Guest 内 vCPU 抢锁超时后，通过 **WFI 陷出到 Host**，Hypervisor 将其移出运行队列。

待锁释放后，Guest 通过 **SMC 调用**通知 Hypervisor 定向唤醒该 vCPU。

验证效果（hackbench 压测、单位s）

PVspinlock	1：1	1：2
ON	4.6205	7.2483
OFF	4.2804	8.2052

1:2 超分场景：

不开 PVspinlock：**8.2052**

开启 PVspinlock：**7.2483**

收益：**11.66%**

结论：开启 PVspinlock 让 1:2 超分场景的完成时间缩短约 10%–12%。hackbench 为高并发进程通信负载，实际业务收益仍需用业务负载测量。

CANN

CONFIG_HZ 校准：并发 resume 平均耗时降低 34%

问题背景

测试项 **concurrent_resume** 是 AgentENV 沙箱快照 resume 路径在高并发下的时延基准测试。

每个测试 sample 并发起 **50 个沙箱**，执行完整快照恢复流程。

openEuler 存在 100ms+ 异常点，方差明显大于 Ubuntu，max 时延约为 Ubuntu 的 **2 倍**。

优化方案

对齐ubuntu，将 **CONFIG_HZ** 从 **250** 改为 **1000**：

HZ=250：每秒 250 次时钟中断，间隔 **4ms**

HZ=1000：每秒 1000 次时钟中断，间隔 **1ms**

原理：沙箱恢复路径上 **ublk_ctl_mutex** 全局锁竞争，抢不到的线程进入睡眠等待被唤醒。但被唤醒的线程与当前运行线程同优先级，通常不会立即抢占，而是要等下一次调度周期重新评估，而 HZ 这个配置就决定了这个睡眠周期。

openEuler 6.6.0 测试结果（50 samples/组，3 轮）

指标	HZ=250	HZ=1000	说明
min（最快 samples）	28.6 ms	24.2 ms	实验的最快 samples 都在 24~29ms，基本是恢复流程本身的成本
mean（平均）	57.6 ms	37.9 ms	HZ 改成1000后，**平均快了约 1/3**
max（最坏 samples）	147.5 ms	108.0 ms	HZ 改成1000后，**最坏异常点缩短了 39.5ms**
>80ms 的 samples 数	13 / 150	1 / 150	HZ 改成1000后，**慢 samples 基本清除**

ubuntu 24.04 对比

mean：53.9 ms → **45.2 ms**（快 16%）

max：121.5 ms → 115.9 ms

>80ms 样本：5 → 0

Ubuntu 在 HZ=1000 下表现更稳定，但优化幅度小于 openEuler。

CANN

目录

Part 1 Agentic RL 介绍和训练流程

Part 2 AgentENV 在鲲鹏 CPU 上的适配与优化

Part 3 DeepSeek-V4 低精度训练与训推精度对齐方案

Part 4 实验、总结与Future Plan

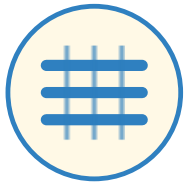
Agentic RL 训练的重点特性



长序列

多轮工具调用
推高激活与 loss 内存峰值

关注容量：Activation、KV Cache、
loss 峰值内存占用



低精度

MXFP8 / MXFP4
降低计算与传输成本

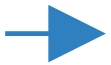
关注效率：计算、存储与传输
量化规则与权重同步

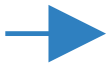


训推一致性

同权重、同输入
得到可信的策略概率

关注可信度：Batch 一致、
Routing-replay、融合 Kernel 一致





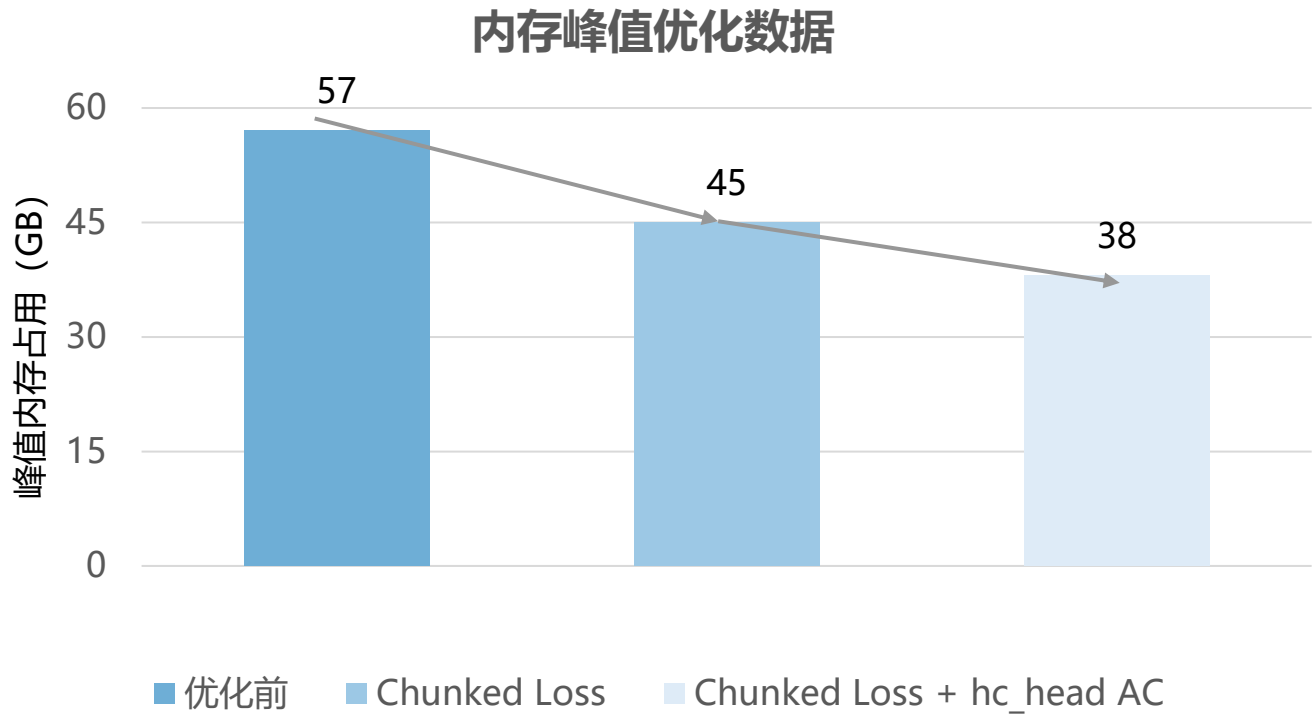
容量 → 效率 → 可信度

三项能力共同决定 Agentic RL 的训练闭环质量

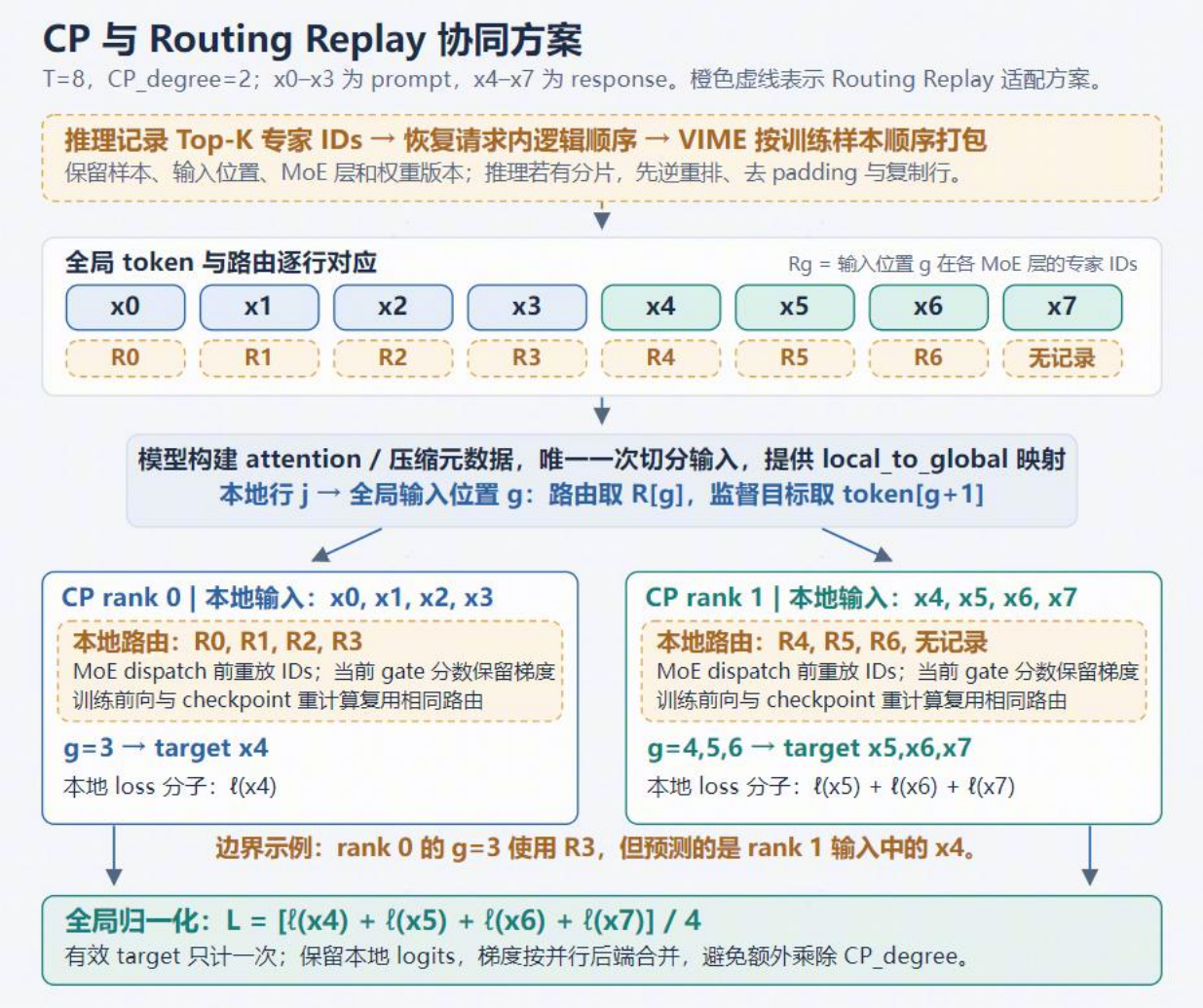
CANN

长序列：内存峰值优化

- **Chunked Loss** 沿 token 做分块处理，在不影响精度的同时**显著降低峰值内存占用**
- **重计算优化** 调整最后一层重计算边界，移出 hc_head，**减少**重计算时的同时驻留张量**内存占用**
- **优化结果** 32K、4P、chunk=8192 场景下：
  如柱状图所示
  从优化前的峰值内存57GB，叠加上述优化后**降低至38GB**



长序列：Context Parallel



CP 将一条长序列**切分到多个Rank计算**，使每个 Rank 主要处理本地 token 对应的激活，在Attention阶段通过**通信获取完整序列的KV信息**。但在RL场景，CP需要处理以下问题：

- **Token 与训练目标对齐**

  logit[g] 预测 token[g+1]，mask 与 advantage 按预测目标分配

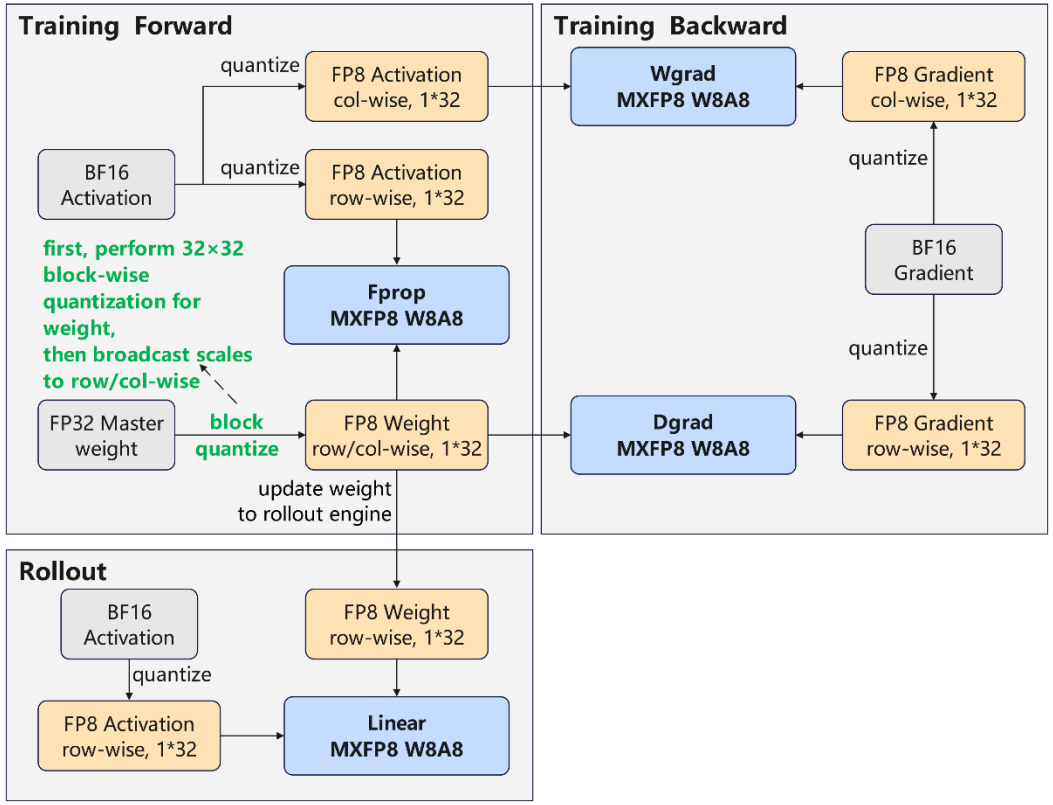
- **Loss 处理**

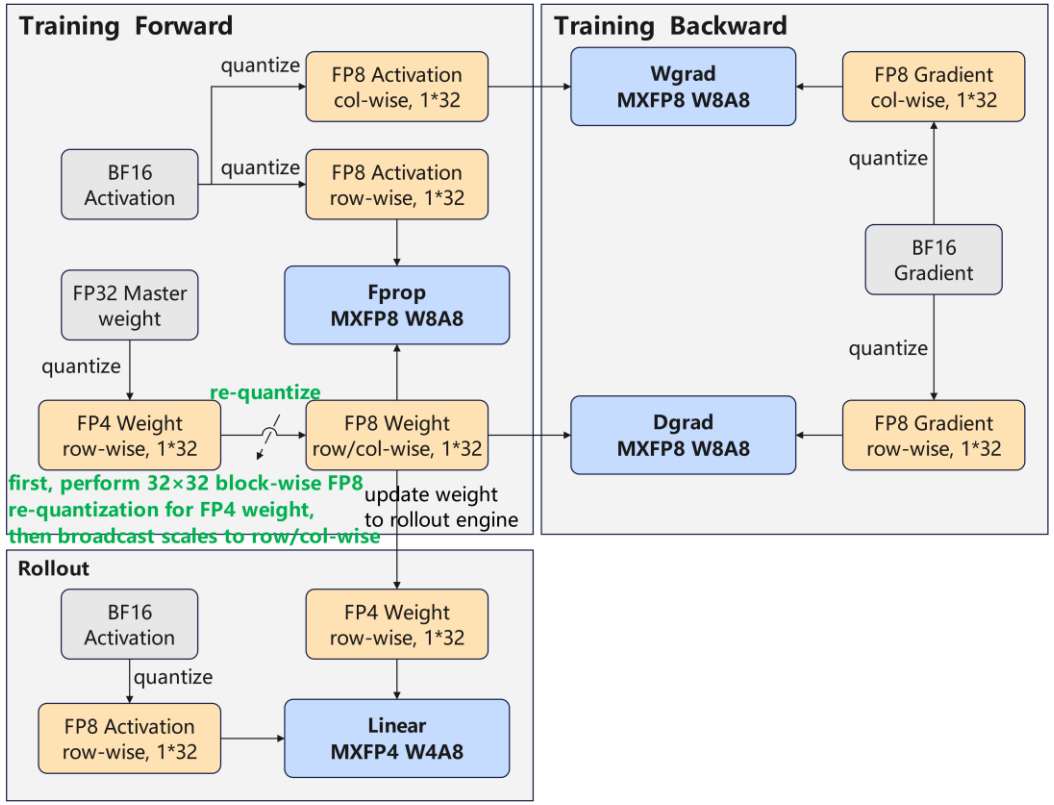
  CP 的多个 Rank 共同处理同一组输入，各 Rank 只需计算本地有效 token 的 loss

- **CP 下的 Routing-Replay**

  CP 需要对应处理 Routed-experts，正确分配每个rank上本地token所需要的专家分布

低精度：MXFP8/MXFP4





为保证 RL 训推一致性，训练阶段与 Rollout 推理阶段采用严格一致的数据类型和量化策略——**FP8 32×32 Per Block 量化。**

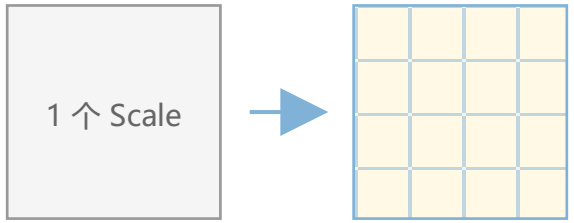
MXFP4相较MXFP8主要**区别**在于：训练正向阶段，FP32 Master Weight 首先量化为 MXFP4 格式，后续重量化为 per-block FP8 权重，用于训练过程中的重计算和反向传播。

低精度：为什么采用 32x32 per-block 量化

权重采用 FP8 32×32 Per Block 量化

01 更细的 Scale，更高的精度

对比 DeepSeek-V4 原生 128×128 粒度



示意：同一片 128×128 权重区域

Scale 更贴近局部权重分布

更细粒度表征 → 提高量化精度

02 权重表示一致，量化结果复用

避免直接 Per-group 量化的轴差异

Per-group 前后向量化轴不同
→ 可能引入权重偏差与 RL 训练稳定性风险

前向生成 FP8 Weight 并缓存
Per-block 权重表示

前向 | 重计算 | 反向

三阶段复用同一份量化权重

前后向权重表示完全一致

减少重复量化开销，支持稳定训练

03 复用 Kernel，发挥硬件效率

适配 Ascend 950DT 的 MXFP8 计算

Block-wise E8M0 Scale

广播适配

MXFP8 Per-group Kernel

L0C 内随路反量化

无需搬至 UB 再由 Vector 反量化

复用 Kernel，无需额外计算路径

避免 UB 容量、Vector 算力与带宽制约

CANN

训推一致：三级递进训推一致对齐

理论上训推使用相同权重情况下，应当计算得到相同的 logprobs，但实际上受到**框架调度、算子调用、Kernel 实现、计算精度、并行策略以及权重处理流程**等因素的影响，训推计算可能会得到不一样的结果，而这部分的差异会**直接影响 RL 训练过程。**

03
RL 整体对齐

RL 场景的训推一致性
同权重、同输入：对齐 Rollout 与 Training 的 logprob
跨框架校准专家路由、数据类型、融合实现与并行计算路径

两侧各自稳定后，再做跨框架比较

02
框架内部对齐
推理、训练分别验证

推理框架内部一致性
vLLM-Ascend
不同 Batch 大小、位置与组合下结果一致
包含 Prefill / Decode 计算结果一致性

训练框架内部一致性
TorchTitan-NPU
不同 Batch 大小、位置与组合下结果一致
前向与重计算保持相同数值与路由语义

共同建立在 Kernel 的 Batch 一致性之上

01
Kernel 基础能力

所有参与训推计算的 Kernel 保证 Batch 一致性
固定输入与权重，改变样本在 Batch 中的位置、Batch 大小或组合，结果保持一致
覆盖计算与通信算子，固定计算及归约顺序，为上层验证提供确定性基础

CANN

训推一致：MoE Routing Replay

01 推理侧捕获

vLLM-Ascend / MoE Router

Router logits 计算 Top-K

捕获逻辑 topk_ids
捕获点位于 EP 本地专家编号映射之前

EP 本地编号映射与专家计算继续执行

还原 token 布局

恢复 DP / TP / SP 改变的 token 排列
裁掉通信或图执行引入的 padding

02 按 token 保存与恢复

每个原始 token 保留一行路由

建立逻辑 token 与路由存储位置的映射
压缩 KV slot 不能直接替代路由 slot

分块保存，覆盖尾部与缓存复用

已有 block：重建非压缩映射，写入 host buffer
未落盘尾部：存入请求级 pending buffer
兼容 Prefix Cache 与 Chunked Prefill

按请求恢复 Routed Experts

按逻辑 token 顺序拼接，去掉填充与重复行
张量布局：[T, L, K] token × MoE 层 × Top-K

03 训练侧回放

用模型的 CP 映射取本地路由

通过 local_to_global 找到本地 token 的全局位置
输入位置 g：取 route[g]
对应 loss 目标：token[g+1]

使用记录的专家 ID 执行同一路径

Replay IDs → Dispatch → Experts → Combine
显式标记缺失记录和末 token，保持 token 对应

比较训练与推理的 logprob

固定权重与输入，先确认专家选择一致
再逐层定位 dtype、融合与 Kernel 数值差异

两处必须保持的数据对应关系

异步调度：每轮 forward 独立快照

先创建 device snapshot，再异步 D2H，避免下一轮覆盖捕获 buffer
物理 block 快照随 SchedulerOutput N 保存，与 ModelRunnerOutput N 对应

Checkpoint：前向与重计算复用同一路由

路由记录绑定 microbatch、层和 CP 布局
原 forward 与重计算读取同一份记录，避免调用顺序变化引起错位

CANN

训推一致：关键问题与解决方案

验证顺序：**01 单引擎内部一致性（含 Prefill / Decode）** → **02 训推引擎间一致性**

	关键问题	差异从哪里来	如何对齐
框架侧	**01 MoE 执行路径不一致** PD 分离 / 混部场景	Prefill：标准 AllToAll；Decode：融合算子 混部时 Decode token 可能走 Prefill 路径 → 同一 token 随 Batch 组合改变计算路径	**统一推理侧 Dispatch / Combine** 全部采用 CANN 融合 Kernel 保持不同阶段、不同 Batch 组合的路径一致
框架侧	**02 数据类型不一致** 权重 + Activation	各模块混用 FP32 / BF16 / MXFP8 / MXFP4 加载、反量化、FSDP 通信、Forward 转换 → 任一环节的类型差异均可能传递到输出	**逐模块、逐环节对齐 dtype** 让训练侧实际参与计算的数据类型 与推理侧保持一致
框架侧	**03 融合粒度不一致** 数学公式相同，数值仍可能不同	推理采用更大粒度融合，训练策略不同 → 中间精度、计算顺序与归约顺序改变	**逐点权衡性能，统一算子实现** 优先：训练复用推理融合算子，并补齐反向 算子不兼容或无反向实现：推理回退到基础小算子
算子侧	**04 物理布局影响计算顺序** CSA Kernel / PageAttention 逻辑 KV、Top-K 顺序均相同	BlockTable 排布改变 → 按物理地址排序搬运两个 token → 偏离逻辑 Top-K 顺序，归约误差变化	**修复搬运逻辑，保持逻辑 Top-K 顺序** 使 Attention 计算顺序不受物理布局影响 消除 BlockTable 变化导致的精度波动

CANN

目录

Part 1 Agentic RL 介绍和训练流程

Part 2 AgentENV 在鲲鹏 CPU 上的适配与优化

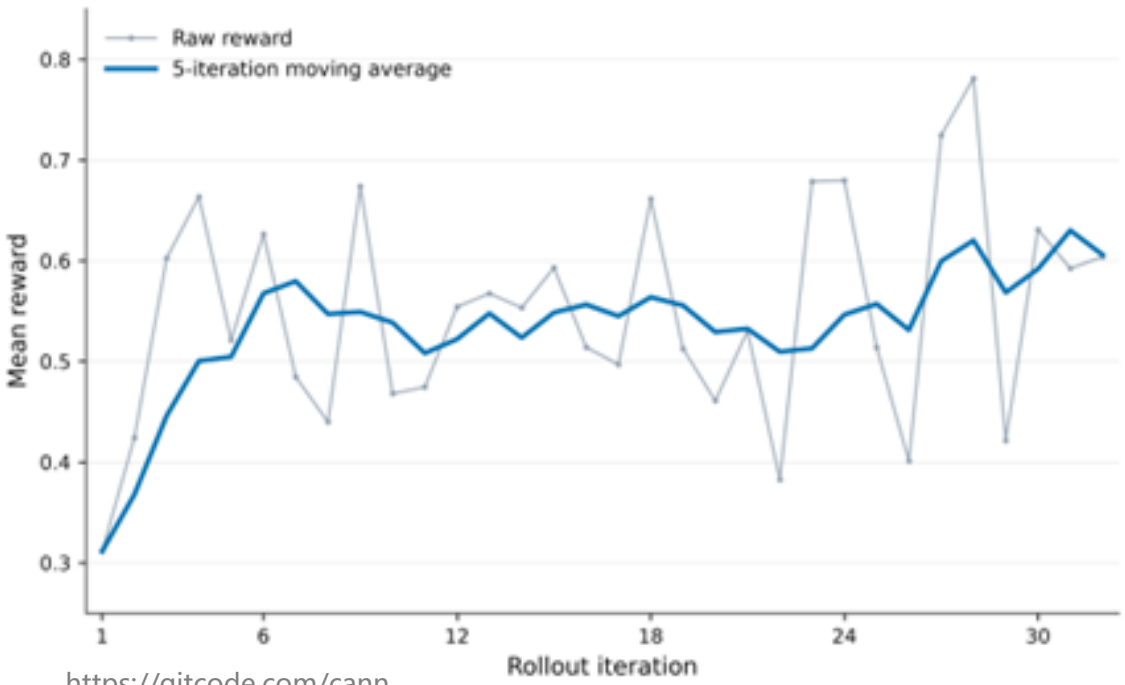
Part 3 DeepSeek-V4 低精度训练与训推精度对齐方案

Part 4 实验、总结与Future Plan

实验

◆ Qwen3.5-4B Agentic

- 任务：SWE-Gym
- 环境：Ascend 950，Kunpeng 950
- 部署：训推分离，各 4 卡
- 参数：batch 16 × 每样本 8 条



https://gitcode.com/cann

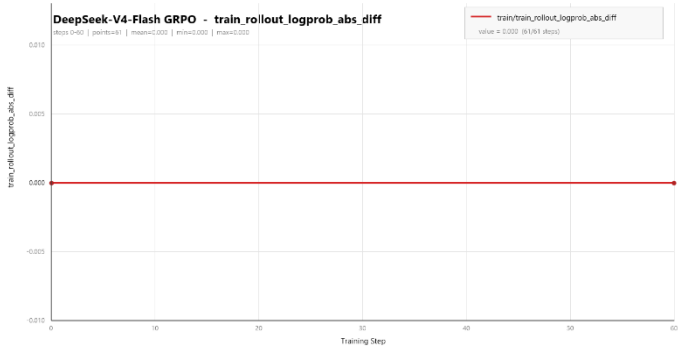
Reward趋势

◆ DSV4-Flash

- 数据集：dapo-math-17k（reward过滤）
- 环境：Ascend 950
- 部署：训推分离最小部署方案
- 参数：batch 128 × 每样本 8 条



Reward趋势



Logprob_abs_diff趋势

总结

- Agentic RL 训练介绍
  - RL 训练基础知识
  - Agentic RL 训练框架和基于鲲鹏+昇腾+灵衢协同的部署方案和计算流程
- 鲲鹏 AscendENV 优化方案：
  - PVspinlock 实践优化：1:2 超分场景性能提升 11.66%
  - CONFIG_HZ 校准：并发 resume 平均耗时降低 34%
- DeepSeek v4 的训练实践
  - 超长序列的内存优化方法
  - 达成训推精度一致的方法

Future Plan

1. 支持 DeepSeek-V4.1 等新模型的 Agentic RL 训练
   - 当前实践仅支持 DeepSeek-V4 Flash 的 Reasoning RL 和 Qwen3.5 的 Agentic RL 训练，后续将持续扩展模型适配范围，支持 DSV4.1 等新一代大模型的 Agentic RL 训练流程。
2. 提升训练稳定性与可靠性保障
   - 开展长稳运行与可靠性测试，完善故障监控、问题定位与恢复机制，保障 Agentic RL 训练稳定可靠。
3. 完善训推一致性问题定位与调试能力
   - 增强精度问题分析与维测能力，围绕训推一致性问题沉淀算法/框架问题定位 Skill，并提供 Kernel 级 Dump 与分析工具，实现问题快速定界。
4. 完善异步 RL 性能分析与优化能力
   - 建设性能问题维测能力，针对异步 RL 训练流程开展端到端效率分析，快速识别 Rollout、通信、训练等环节瓶颈，持续优化系统性能。
5. 完善 PD 分离场景下的 Routing Replay 实现
   - 针对 PD 分离部署场景，完善 Routing Replay 机制，保障 PD 分离模式下训推路由一致性，提升长上下文 Agentic RL 场景的稳定性与训练效率。
6. 提升 Agentic RL 任务的规模化和丰富度
   - 当前版本主要展示了 SWE-Gym 的训练效果，之后本项目将聚焦“多任务 + 多 Harness”的 Agentic RL 训练场景。基于已有的稳定训推精度和轻量解耦的环境，本工作将继续演进并提升模型后训练能力的泛化性。

欢迎体验和贡献

cann-recipes-infer

cann-recipes-train

cann-recipes-infer:
https://gitcode.com/cann/cann-recipes-infer

cann-recipes-train:
https://gitcode.com/cann/cann-recipes-train

cann-recipes-sig小组

cann-recipes交流群

欢迎广大开发者体验并参与贡献，如有疑问也可通过issue、SIG或者cann-recipes交流群联系我们！

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann